Crystal Tier List Bot

Full installation and operations guide

Version 0.1.0 | Prepared October 11, 2026

This package contains a configurable Discord bot for Crystal PvP account linking, regional testing, tier lists, support tickets, deadlines, restrictions, and backups. Each installation serves one Discord community using one bot application and one running process.

The full-package ZIP includes compiled JavaScript, TypeScript source, automated tests, pinned dependencies, configuration templates, SQL setup, launchers, and this guide. Node.js and downloaded dependencies are not bundled. Internet access is needed for installation and normal operation.

Release status: 27 local automated tests passed. A clean Windows installation passed configuration validation. Discord behavior is tested with simulated transports; PostgreSQL uses an emulator and Supabase uses a mocked API in these checks. Live Discord, real remote databases, and clean Linux/macOS installation still need rehearsal. Use an isolated Discord server first.

Read the quick start for the shortest installation route. The later sections explain configuration, permissions, hosting, recovery, migration, and troubleshooting. Commands shown in this guide must be entered in the specified place: host terminal commands on the machine running the bot, and slash commands inside your Discord server.

Repository: https://github.com/raincakecat/TierList-bot

1. Quick start with SQLite

SQLite is the easiest first rehearsal because it runs on your host without a separate database service. Supabase remains the database selected in the template, so explicitly change that setting if you choose SQLite.

1. Install Node.js 24.17.0 or newer from the official Node.js download page. Open a new terminal and check both tools:

```
node --version
npm --version
```

2. Extract full-package.zip. Open a terminal inside the extracted full-package folder: package.json should be directly in this folder. Keep it on a local writable disk.

3. Install only the runtime dependencies:

```
npm ci --omit=dev
```

4. Create your own configuration and credentials files. Windows PowerShell:

```
Copy-Item config.example.json config.json
Copy-Item .env.example .env
```

Linux or macOS:

```
cp config.example.json config.json
cp .env.example .env
```

5. Edit config.json: set community.guildId to your server ID in quotation marks; set database.backend to "sqlite"; choose community.name and timezone; set testing.normalCooldown and testing.highCooldown to "0" for a rehearsal. Leave resource IDs null if setup should create them. Leave testerQuotas and migration disabled initially.

6. Create and invite a Discord bot as described in section 2. Put its token after DISCORD_TOKEN= in .env. Move its server role above all roles it will assign.

7. Start it with npm start. In Discord, run /setup preview, then /setup apply and confirm. Review /config validate, fix any errors, and use /panels refresh. Initial setup publishes panels automatically when all checks pass.

Before automatic setup, npm run validate can report missing role/channel IDs. That is expected for a template with null resources. Cooldowns and your server ID must still be explicitly chosen. The bot can connect for setup before the configuration is ready for user requests.

2. Create and invite the Discord bot

Create an isolated Discord server that you control. In Discord settings, enable Developer Mode and use Copy Server ID on that server. Put the value in config.json as community.guildId. A server ID is different from the bot application ID.

Open https://discord.com/developers/applications and create an application. Use its Bot page to generate a bot token. Save the token only in your local .env file. Regenerating a token invalidates the old one, so update the host file and restart afterward.

This bot connects to Discord's Gateway. It does not need an Interactions Endpoint URL, a public web server, ngrok, an application public key in .env, or inbound firewall ports for Discord interactions. It needs outbound network access.

Enable these three privileged intents on the application's Bot page:

- Server Members Intent: membership, roles, and user access checks.
- Message Content Intent: ticket transcripts and evidence links.
- Presence Intent: opponent availability checks.

Choose Guild Install for the installation context. In the installation settings or OAuth2 URL generator, include the bot and applications.commands scopes. Open the generated invite link and select your test server.

Grant the bot Manage Channels, Manage Roles, View Channel, Send Messages, Read Message History, Attach Files, Embed Links, Add Reactions, Manage Messages, and Mention Everyone. The last permission supports regional availability announcements; review it deliberately. Apply the necessary permissions in each mapped channel as well.

In server role settings, move the bot's highest role above Owner, Bot Admin, Staff, tester, region, waitlist, and tier roles that it needs to manage. Never map @everyone or a managed integration role as an assignable role. Discord Administrator permission is not required on the bot itself; the first human setup operator needs it.

Private tickets and archive channels must deny ordinary members access. Test the final channel permissions using an ordinary member account. Existing channel IDs are reused, so their privacy settings need manual review too.

3. Configuration files and placeholders

config.example.json is a template. config.json is your installation's live settings. The schema is strict: do not delete required sections or add unknown keys. JSON has no comments. Write IDs as quoted strings; unquoted Discord IDs can lose precision. null means that a resource has not been supplied. Arrays of permission roles start as [].

community: set guildId, a community name, and an IANA timezone such as America/New_York or Europe/London. Statistics and monthly reports use this timezone.

modules: testing, supportTickets, and tierLists can be enabled separately, but tierLists and testerQuotas require testing. The tierLists switch covers all five HT3, LT2, HT2, LT1, and HT1 lists. Migration is optional. Disabling requests does not erase existing tickets or history.

regions: NA, EU, and AS each have enabled, regionRoleId, waitlistRoleId, queueChannelId, and testCategoryId. Enable at least one region for testing. A region role records the selected region; a separate waitlist role grants regional queue access.

tierRoleIds: map all ten LT/HT tier roles if testing is enabled. Do not reuse a Staff, Owner, or region role as a tier role. testing.queueCapacityPerRegion defaults to 20. Choose cooldowns explicitly: "0", "30m", "3h", or "7d". null cooldowns must be resolved before requests are enabled.

channels: map verificationPanel, supportPanel, results, testerAlerts, testingArchives, quotaReports, staffActionLog, operationalLog, and botReview as required. categories.highTests is a category ID. categories.overflow is an ordered array of category IDs. Use category IDs for categories and text-channel IDs for channels.

supportTicketTypes: each type has a unique key, enabled flag, display name, category, archive channel, access/closing/ping groups, evidence-copy flag, and opening message. Use lowercase keys such as general-support. An enabled type needs its category and archive destination.

branding.colors supplies five hexadecimal colors. Message templates use Discord Markdown and placeholders such as {communityName}, {userMention}, {minecraftIgn}, {discordUsername}, {region}, {previousRank}, {earnedRank}, {preferredServer}, and {previousTest}. Unknown placeholders fail validation. Do not assume every placeholder has a meaningful value in every message.

archival sets directory, image retention, and deletion delay. Transcript retention stays null: transcripts do not expire automatically. backups defaults to daily with ten successful automatic backups retained. Keep storage folders writable and private.

4. Choose and configure a database

Use one backend per installation. Do not run two bot processes against the same community database. The saved state includes account associations, ranks, restrictions, queues, tickets, jobs, and settings needed for recovery; transcripts live separately on disk.

SQLite - local and simplest

Set database.backend to "sqlite". The default database.sqlite.file is ./data/crystal-bot.sqlite. No database credentials are needed in .env. The host must be able to create the data directory. Keep the database on a local disk, not a shared network filesystem. On Node 24, an experimental SQLite warning can appear even when the bot works correctly.

Supabase - hosted PostgreSQL through the API

Create a dedicated Supabase test project. Run sql/supabase.sql once in its SQL editor. This creates the bot state table, enables row-level security, and installs the atomic commit function. Populate .env with the project's API URL and server-only service_role key:

```
SUPABASE_URL=https://YOUR_PROJECT.supabase.co
SUPABASE_SERVICE_ROLE_KEY=YOUR_SERVER_ONLY_KEY
```

The environment variable names come from database.supabase.urlEnv and serviceKeyEnv. The template uses the legacy service_role key name; do not substitute the browser anon key or a publishable key. Service credentials bypass RLS and belong only on your private host. New Supabase secret-key formats need a dedicated live compatibility check for this candidate build.

PostgreSQL - direct database connection

Set database.backend to "postgresql" and configure DATABASE_URL in .env. The database must already exist, and its account must be allowed to create/use the crystal schema and state table. The bot creates that schema/table on startup. Use your provider's verified TLS configuration for remote connections.

```
DATABASE_URL=postgresql://USER:PASSWORD@HOST:5432/DATABASE
```

Encode special characters in connection-string credentials. Do not paste real connection strings into public issues or the guide. SQLite state can later be transferred through the Owner backup tools; test any transfer using disposable data first. No Supabase or PostgreSQL server is bundled in this ZIP.

5. Role groups and resource mapping

Permission groups are arrays of Discord role IDs, not member IDs. A group may include more than one role. The configured Owner group can use the bot's other permission groups. Other groups should be mapped deliberately; having Discord Administrator permission is only the initial setup fallback, not a replacement for routine bot role mappings.

- owner: setup/configuration, maintenance, exports, restore/transfer, migration apply/retry, player resets, and bulk actions.
- botAdmin: panel refresh, unlinking accounts, tester assignment, cooldown settings, quotas, and supported Bot Review corrections.
- staff: ticket participant tools, renaming, testing results, evaluation passing, fight templates, ticket registration/conversion, and statistics.
- verifiedTester: tester sessions and taking the next queued test. Testers also need Staff access.
- regulator: deadlines, forced test creation, forced results, rank migration, and forced regional queue stops.
- listAdmin: adding/removing high-list members. Staff can view lists.
- restrictionKey: applying and removing account-pair restrictions.
- highStaff: access/pings/closing for sensitive support types in the default configuration.
- trainee: the default general-support opening ping group. Map access separately if those members should read that ticket type.
- partnershipManager: the default partnership ticket access/ping/closing group.

Type-specific support permissions matter: Staff does not automatically gain access to sensitive highStaff tickets. Add the appropriate groups to that type's accessGroups and closingGroups if your policy differs. A ticket tool also requires the invoking member to see the ticket.

For a fresh server, leave role arrays empty and resource IDs null. Setup creates missing resources and writes the generated IDs into config.json. Supply existing IDs to reuse your own channels, categories, or roles. Back up config.json after setup because it now contains the mapping for your server.

Assign the created roles to the right human operators. /addtester assigns both the configured tester and Staff roles. /stoptester removes tester authorization and the active session; it does not automatically remove Staff.

The bot creates new private channels according to ticket type. When a category fills, overflow uses the ticket's access rules. Explicitly check any existing overflow categories and sensitive archive destinations before a live rollout.

6. First startup and setup confirmation

From the installation folder run npm start. You should see a successful connection message. Commands are registered for the configured server, not globally. Use the application command picker inside that server.

1. As a Discord Administrator in an uninitialized server, run /setup preview. Read the resources it intends to create. Supplied IDs will be reused. 2. Run /setup apply and confirm as the same operator. Confirmation is tied to the operator and expires after five minutes. Preview again if the configuration changed or the confirmation expired. 3. On first successful setup, the confirming operator receives the configured Owner role. Each created resource is saved immediately, so a failed setup can continue without recreating everything. 4. Run /config validate. Resolve missing settings, role hierarchy, channel types, and read/send/archive permissions. If setup reports unresolved settings, edit config.json and repeat setup or reload after fixing them. 5. Assign staff groups and /addtester as needed. Confirm private access with ordinary members. 6. Use /panels refresh to publish or update buttons, menus, regional queues, and tier lists. Automatic setup also publishes them when validation succeeds.

npm run validate runs on the host and checks JSON structure and required settings; it does not contact Discord. /config validate checks actual Discord resources as well. A clean local validation is necessary but does not prove live channel privacy or network behavior.

For configuration changes after setup: use /maintenance enabled:true before changing roles, channels, categories, or modules. Edit the live JSON, run /config reload as Owner, then /panels refresh. Disable maintenance only after the new mapping validates. Tester sessions must be started again.

Database backend or community server changes require stopping the bot and planning a transfer; /config reload will not switch those live. Template, color, cooldown, and testing settings can be updated without changing the database.

Always run from the installation folder. Relative paths for .env, config.json, archives, and backups are resolved from the process working directory. If a host panel lets you choose a working directory, point it to the folder containing package.json.

7. Player verification and testing workflow

Verify Account requests the Minecraft in-game name, one enabled region, and preferred server. The Minecraft lookup validates an existing username and UUID; it does not prove account ownership. One Minecraft UUID can be linked to one Discord player at a time. Verification assigns the ordinary region role.

Enter Waitlist gives the separate region waitlist role and access to that region's queue. Verification alone does not grant this waitlist role. Members cannot change region themselves by verifying again. Bot Admins use /unverify when an incorrect/lost Minecraft account needs to be unlinked; this preserves Discord rank and cooldown, clears the association/region/waitlist, and allows re-verification. Open tests must be finished first.

For a normal test, a tester uses /start region:NA (or EU/AS). Eligible players press Join Queue in their regional panel. /next reserves the earliest eligible entry and opens a ticket for the tester. /leave removes a player from the queue. /stop ends the tester session; the last tester stopping clears that regional queue. Restart also ends active tester sessions.

Normal evaluation tickets open in their regional category as test-username-region. Opening messages mention the player and assigned tester and show their saved account information. Previous Rank is the rank captured when the ticket was opened, even if the player receives an interim rank afterward.

/passeval records LT3, moves the ticket to High Tests, and renames it username-ht3-region. After evaluation passes, /skip is unavailable. Complete it with an actual result or Test Discontinued. Discontinuation restores the opening rank and does not count a completed test.

Existing high-tier players bypass the normal queue: HT3 requests LT2; LT2 requests HT2; HT2 requests LT1; LT1 requests HT1. Staff add a tester with /add in an unassigned high test. Adding a member sends a mention. HT1 requests are directed to general support for rank defenses or retirement.

/close records the chosen tier, updates roles and eligible lists, credits the original tester once, applies cooldown, posts the result with six reactions, and queues archival. /format posts an editable Markdown fight template; it does not decide the result.

HT3 list membership is manual. LT2 and higher membership is maintained automatically for eligible linked players. List embeds are grouped by region. Restrictions remove list memberships and block testing; expiry alone does not restore those memberships.

8. Command reference: players and ticket staff

Commands may be visible in Discord before a user is authorized. Runtime checks enforce the groups below. Use the command picker for the exact required options. Owner inherits group authorization; ticket visibility checks still apply.

Member commands

- /profile [member]: view a saved player profile; defaults to yourself.
- /leave: leave your regional queue.
- Verification, waitlist entry, Join Queue, and support opening use buttons/menus rather than separate slash commands.

Verified Tester commands

- /start region: start your regional session; /stop ends it.
- /next: reserve and open the next eligible normal test. Testers need the Staff role too.

Staff commands inside tickets

- /passeval: pass a normal evaluation and continue as HT3 testing.
- /skip: skip an evaluation before it passes; archive before deletion.
- /close tier: finish a test with a tier or Test Discontinued. Support tickets use their close button and responsible staff groups.
- /format tier: post a fight result template.
- /add member and /remove member: manage added participants; required role-based access and the opener/original tester are protected.
- /rename name: change the ticket channel name.
- /addht3 and /addlt2: select an eligible opponent from the regional list.
- /set-ticket type member [tester]: register an existing channel. A test requires the original tester. Check metadata before adoption.
- /changeticket type [tester]: change a support route or convert to testing/high testing where permitted.
- /uuid ign: staff-only Minecraft UUID lookup.
- /stats [member], /testerstats, and /testerlb [period]: tester counts, quota statistics, and monthly/weekly/all-time leaderboard choices. /stats and /testerstats currently use the monthly period.

List commands

/ht3list, /lt2list, /ht2list, /lt1list, and /ht1list each have list, add, and remove subcommands. Staff can view lists; List Admins can add/remove with an explicit region. Adding requires the player's saved rank to match and the account to be unrestricted.

9. Command reference: administration

Bot Admin

- /unverify member: unlink the Minecraft account while preserving Discord rank/history and cooldown.
- /addtester member and /stoptester member: assign/remove tester authorization and manage active sessions.
- /resetcooldown member and /setcooldown member days: clear/set an individual cooldown.
- /configcooldown kind duration: change future normal/high completion cooldowns; "0" disables that cooldown.
- /configquota count: set the monthly quota; 0 disables quotas. Enabling it requires the quota report channel to be configured first.
- /panels refresh: update the configured request panels, queues, and lists.
- /migration preview filename: inspect an enabled legacy import without applying it. Bot Review corrections require Bot Admin.

Regulator

- /deadline set duration or /deadline clear: use 1m through 14d, including 3h or 2d. Expiry mentions the setter for review; it does not close the ticket.
- /forcestopqueue region: stop regional tester sessions and clear their queue.
- /forcetest member region: bypass queue/cooldown; restrictions and duplicate open tests still apply.
- /forceresult member tier region [ign]: assign a result outside an active test.
- /migrate member tier region source [ign]: assign a rank from another community. This is separate from the bulk legacy /migration feature. Both rank commands require an explicit region; unverified users need a valid IGN.

Restriction Key

/restrict member ign reason [duration] restricts the Discord/Minecraft pair. Omit duration for indefinite; 0 is not an indefinite restriction duration. /unrestrict member removes that pair restriction. Restricting removes queue/list membership but does not automatically archive an existing test.

Owner

/setup preview|apply and /config validate|reload manage setup/configuration. /maintenance enabled controls maintenance. /backup export|restore|transfer handles saved state. /migration apply|retry applies/resumes imports and retries saved failed work. /reset member clears that player's account/rank/queue/cooldown while preserving restrictions and archives. /forceresetcooldowns clears all cooldowns. /forceclosetickets discontinues and archives active open tickets. Bulk/reset/restore/apply operations use review confirmations; inspect them before approving.

10. Support tickets, evidence, and deadlines

The defaults are General Support, Player Report, Staff Report, Restriction Appeal, and Partnership Request. Add custom support types by following the existing object structure in config.json. Every enabled type needs a valid category and archive text channel.

accessGroups determines which staff roles can view the type. openingPingGroups determines the roles mentioned at opening. closingGroups determines which staff can use its Close button. The opener is mentioned and receives private ticket access; the opening message is pinned. Configure ping groups with access if they should respond inside the ticket.

Closing locks the ticket and queues asynchronous work. The durable queue saves a Markdown transcript and any required images before posting the archive and deleting the channel. If required evidence fails, the channel remains for review and jobs retry with backoff. A delay of at least ten seconds separates closure processing from deletion.

copyImages defaults to true for reports and appeals. It copies supported images from Discord attachment hosts, with a 25 MiB per-image limit. Videos are not copied. Attachment links and YouTube URLs remain in the transcript. A link may later expire or become unavailable, so this is not a permanent video evidence store.

Images are saved privately in the archive directory and removed after the configured imageRetentionDays (default 10). Transcripts do not automatically expire. The current archive attachment limit is 8 MiB per Markdown transcript; larger required transcripts retain the channel for staff intervention. Monitor available disk space.

Use /deadline set duration:3h as a Regulator who can access the ticket. It sets/replaces the deadline and notifies participants. /deadline clear cancels it. At expiry, the person who set it receives the review notification. It does not delete the ticket, override evidence requirements, or choose a result.

Ordinary members should not see staff reports, restriction appeals, archive channels, Bot Review, operational logs, or staff action logs unless your deliberate policy grants access. Review this with an ordinary account; Discord administrators can bypass normal channel privacy, so an administrator account is not a useful privacy test.

11. Run continuously on a Linux VM

Use a supported Linux distribution with Node.js 24.17.0 or newer installed through a trusted installation route. Verify the actual installed version; a distribution's default package may be older. Extract the full package into a persistent application folder and install runtime dependencies there.

Test npm start in the foreground before enabling an automatic service. Stop it with Ctrl+C. Use one process only. A saved process lease blocks concurrent instances; after an unclean exit, wait up to 60 seconds before another instance can acquire the database.

For systemd, create a service under /etc/systemd/system/crystal-bot.service. Replace the example user and paths with real values. Do not put credentials into the service file. Use an ordinary dedicated account that owns the application/data folders.

```
[Unit]
Description=Crystal Tier List Discord Bot
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=YOUR_LINUX_USER
WorkingDirectory=/opt/crystal-bot
ExecStart=/usr/bin/node --env-file=.env dist/src/main.js
Restart=on-failure
RestartSec=65
TimeoutStopSec=300
UMask=0077

[Install]
WantedBy=multi-user.target
```

Use command -v node to find the actual Node path. A user-installed Node may be elsewhere. The configured user must be able to access it even outside your interactive shell. Review the service file, then run:

```
sudo systemctl daemon-reload
sudo systemctl enable --now crystal-bot
sudo systemctl status crystal-bot
journalctl -u crystal-bot -n 100 --no-pager
```

Store .env with owner-only read/write permissions. On Linux, chmod 600 .env config.json is appropriate when your service account owns them. Maintain host updates, outbound HTTPS, DNS, and database connectivity. No inbound port is needed for this Gateway bot. Do not run PM2 and systemd as competing bot processes; choose one supervisor.

12. Backups, restores, and safe updates

Automatic database exports run daily and retain the latest ten successful automatic backups. Manual exports remain until you remove them. /backup export writes an export on the host; it does not send secrets or create a downloadable Discord attachment. The default folder is data/backups.

Portable exports contain saved bot state and may include member/account information. They exclude environment credentials and runtime leases. They do not contain image/transcript files, your live config.json, or .env. Back up those separately through an authorized private host backup method.

Before an update, create a manual export, preserve config.json and .env, and make a separate backup of data/archives. For SQLite, also preserve the live database safely; prefer a clean stop before copying database/WAL files. Do not overwrite your data folder with a fresh template.

Restore rehearsal: enable /maintenance enabled:true, place a valid backup in the configured backup directory, then use /backup restore filename:YOUR_BACKUP.json. Read the preview and confirm. The current state is backed up first; maintenance remains enabled after restore. Verify saved ranks, tickets, restrictions, and associations before resuming.

Discord channels and messages are external resources. Restoring an old database does not recreate already-deleted Discord channels or undo posted messages. Do not restore an old snapshot to a live community without checking those differences. Live restore races and external side effects still need rehearsal in this candidate release.

Backend transfer: prepare a destination configuration file on the same host pointing to an empty destination database. Use /backup transfer config:DESTINATION_CONFIG.json as Owner in maintenance. The transfer verifies destination state and leaves source maintenance enabled. Stop the source process before changing the active config and starting the destination. Never run both.

For source updates: stop the service, get the reviewed new files, run npm ci, run npm test, and restart only after build succeeds. For full-package installs, replace compiled/source files from the new package but preserve all live settings and data. If you only install runtime dependencies, do not run npm test or npm run build until you install dev dependencies with npm ci.

Review operational logs for pending archive failures and disk pressure. Preserve evidence folders independently of the state backup. Test recovery using disposable tickets before relying on it in production.

13. Optional takeover from an existing bot

Migration is optional and disabled initially. You can use this bot for a fresh community without migrating anything. A takeover should start with an isolated rehearsal and a verified export of the old bot's actual database. Screenshots alone are not a complete database export.

Stop old database writers before the final export/import. Keep original exports unchanged in private storage. Convert a working copy to the crystal-legacy version 1 JSON format described in docs/MIGRATION.md. The new importer does not connect to arbitrary legacy databases or guess missing ranks.

Keep existing Discord channels for adopted tickets. Supply their real opener, original tester, opening rank, region, existing name, and explicit participants. For an evaluation that passed to HT3, the opening rank is the rank before the evaluation ticket opened, not the interim LT3 rank. If unknown, set openingTier to null for review.

Place the converted file under data/imports on the bot host. Enable the migration module through a reviewed configuration change. In maintenance, run /migration preview filename:legacy.json. Fix global errors such as a mismatched server before applying.

Owner /migration apply imports ready records after confirmation and saves a backup first. Unresolved records go to Bot Review. Bot Admins can correct supported missing fields with the provided controls; conflicts involving multiple records require editing the host file. Preview and resume the same file to apply corrected records without repeating completed ones.

A completed import is one-time. /migration retry reschedules saved failed work rather than repeating imported records. Maintain the same import identity and do not invent a new file/session to force a duplicate takeover. Archive access and actual channel existence need checking during rehearsal.

Plan a cutover window: announce it, stop old queue/ticket creation, complete/export pending state, import and validate, publish the replacement panels, and test access using ordinary members. Leave maintenance enabled until all review items are understood. Old messages authored by another bot cannot simply become new bot buttons; publish the new panels and controls.

This guide does not authorize deletion of a community's old tickets or original database. Preserve them until the operators have verified the new bot's data and agreed their retention policy.

14. Troubleshooting

Node/npm not found: install Node, reopen the terminal, and check node --version and npm --version. On Windows, if PowerShell blocks npm.ps1, use npm.cmd rather than weakening machine-wide script policy.

Missing dist/src/main.js: you downloaded the source archive rather than the full package, or extracted into the wrong folder. From source, use npm ci and npm run build. The full package already contains dist/src/main.js.

Create config.json error: copy the template, use valid JSON, keep required keys, and quote IDs. On Windows ensure the actual extension is .json rather than .json.txt. Run npm run validate to check the file.

Missing settings before setup: null role/channel placeholders are expected in a fresh template. Choose the guild, database, regions, and cooldowns, connect the bot, then use setup. Missing cooldowns are not auto-selected.

Discord login failed: check the Bot token, all three required intents, and network access. Use the application bot token, not your personal Discord token. If you reset it, update .env and restart.

Slash commands not visible: confirm the configured server ID, the invite's applications.commands scope, that the bot connected, and that you are looking in that server. Reopen Discord's command picker. This build registers guild commands.

Role must be below the bot: move the bot role above the role being assigned. Do not map managed roles or @everyone. Channel permission errors: verify text-channel IDs, category IDs, and the bot's explicit access/attach/embed/history permissions.

Permission denied: give the operator a role mapped to the command's bot permission group. For ticket commands, also confirm ticket visibility. A Discord Administrator without a configured Owner role is not the normal post-setup authorization path.

Another instance already running: stop competing services/processes. Wait up to 60 seconds after an unclean exit. Do not remove the lease simply to run two processes.

Ticket remains closing: inspect operational logs, disk space, archive channel permissions, image download failures, and transcript size. Failed required evidence keeps the channel. Do not delete it manually before resolving evidence retention.

Supabase failures: confirm the API URL, server service_role key, executed SQL setup, and a dedicated test project's grants. PostgreSQL failures: confirm the database exists, connection string/TLS, and schema creation privileges. Treat remote backends as unverified until a real test passes.

After restart, queues show no testers: sessions intentionally reset. Have authorized testers use /start again. Configuration reload rejected: use maintenance for resource/module changes; stop/restart for database/server changes.

15. Rehearsal checklist and package contents

Before using the bot with a live community, verify:

- Setup preview/apply creates the intended resources and can resume after a partial failure.
- An ordinary member cannot see private tickets, sensitive archives, Bot Review, or staff logs.
- Valid IGN lookup succeeds; invalid IGN, duplicate UUID, duplicate verification, and disabled regions are rejected.
- Verification grants a normal region role; Enter Waitlist grants the separate waitlist access role.
- Two testers cannot reserve the same FIFO entry. Last tester stopping clears the queue without losing open tickets.
- Evaluation passing changes rank/category/name and prevents skipping; discontinued tests restore opening rank.
- Direct high-tier progression and tester assignment work; results use opening rank and add all six reactions.
- Sensitive support types respect access/closing groups even in overflow.
- Deadlines notify the setter once without closing. Restrictions clear queue/list membership and block testing.
- A failed archive retains its channel; successful archival saves transcript/evidence before deletion. Restart resumes saved work.
- A disposable backup restore and optional migration preserve intended records without duplicate credit/imports.
- The chosen real database and target Linux/macOS host pass the appropriate installation/recovery checks.

Inside full-package.zip:

- Full-Setup-Guide.pdf and docs/: setup, migration, and release-status instructions.
- dist/src/: compiled runnable JavaScript and source maps.
- src/, test/, tsconfig.json: source and automated verification.
- package.json and package-lock.json: scripts and pinned dependencies.
- config.example.json and .env.example: blank settings and credentials templates.
- sql/supabase.sql: server-only Supabase state schema and commit function.
- start.bat and start.sh: start the installed runtime from its own folder.
- scripts/: reproducible package-generation helpers.
- MANIFEST.sha256: hashes of package files for integrity checking.

Local credentials, real server mappings, databases, evidence, node_modules, and private host settings are excluded. Install dependencies after extracting. The public repository has no selected project license; review that separately before redistributing it as your own product.

16. Sources and useful links

The instructions for this particular bot were checked against the included source and configuration schema. The following official pages explain the external services; their interfaces can change.

Project source and downloadable full package: https://github.com/raincakecat/TierList-bot

Discord bot application, installation scopes, and token setup:
https://docs.discord.com/developers/quick-start/getting-started

Discord Gateway and privileged intents: https://docs.discord.com/developers/events/gateway

Discord Developer Portal: https://discord.com/developers/applications

Node.js downloads: https://nodejs.org/en/download

discord.js runtime documentation: https://discord.js.org/docs/packages/discord.js/main

Node.js SQLite API: https://nodejs.org/api/sqlite.html

Supabase server/browser API key distinctions: https://supabase.com/docs/guides/getting-started/api-keys

Supabase RPC and row-level security: https://supabase.com/docs/reference/javascript/rpc
https://supabase.com/docs/guides/database/postgres/row-level-security

For exact local behavior, consult src/config.ts, src/domain/service.ts, src/discord/runtime.ts, and docs/STATUS.md. This guide documents the 0.1.0 candidate; repeat validation after changing code, dependencies, roles, or databases.